Agentic Engineering: Building
choose the model. The claim
controls the flow? Who decides
between, an explicit skeleton with
after another. Use it when the steps
triage step that sends a ticket to the
draw the graph, write it in code. The
tasks, checks the results, and retries
maximum number of handoffs) and end in
task. ## 4. State: give each agent its own
They never write over each other. - Claims.
5. Memory and context An agent does not
context is gathered per turn: at the start of
decisions conflict. Persist the facts that must
to the task, not the task to the model. - Start with
normal. - When quality is subjective (a design, a
Measure the choice: compare quality, tokens, and
autonomy. The answer shows up only by trying.
intelligent joints, which is cheaper, more reliable,
or a tool. - Independent subtasks (summarise
the route. - Open-ended work with an unknown
hands out, then checks and retries; guard its
models or prompts, then a human or a judge picks
that dispatches to several workflows. Start
looks like, and include the awkward cases, the inputs
known-bad examples, or it will quietly drift. 3. Run
own test jobs. ## 9. Optimisation: where most of the value is
What each agent can actually do. The right tool, well described,
or try several (section 6), before you add structure. 5. Only then,
task, fetch what you need. - Take action: the model requests tool
"max turns exceeded" rather than loop forever. - Plan between calls:
together. It is an example of the general shape, not the only one. One
automatic. - Report block: an agent gathers sources, does the work, and
it. - Fan-out: split the work into several isolated sandboxes, each running
status, so you can see what each agent did and why. ## 12. Build
write the route in code. 5. Find the one step that cannot be pre-scripted.
artifacts. 7. Choose the model per step: cheapest
gate before anything irreversible; treat tool
"You are {role}. Goal: {task}. Tools available: {tools}.
are a reviewer seeing this work for the first time. The
work; only judge it." Router (a conditional step): "Read
Assign each task to one worker. After each worker

Multi-Agent Systems A
throughout: these systems are not
what happens next. 2. How do agents share
intelligence at the joints. Name where each part of
are known and ordered: draft, then check, then
right handler. Frameworks such as LangGraph express
model is a node, not the orchestrator. ## 3. Autonomous
failures, like a project manager. Risk: the lead is a single point
a clear failure instead of forever. ### Group chat All agents
copy The clash: two agents both write the same file, guide.md.
Each unit of work is claimed by exactly one agent (Task 1 to A,
remember on its own between runs; it gathers the
each run the agent fetches the task, the relevant files,
survive; recompute or re-fetch the rest. - Each
the smallest, cheapest model that passes your
draft), run several models on
calls across models on your
And the rule that matters
and easier to debug than autonomy everywhere. ### By use
many files, grade many answers, enrich many records): a
path (research a question, fix a bug, investigate a system): an
single point of failure. - A chain of specialists (intake, then
the best. - A deliverable that must be correct (a legal brief, a
from the case closest to yours, build it explicitly, and add
that are easy to get wrong. 2. Grade two ways. Use code when
he simplest version alongside the complex one. On
When the system works but is not good enough, optimise in
beats a clever plan over the wrong tools. 3. Context and
more agents. Extra agents and steps are the most
calls; you execute them and feed the results back. - Run
for hard tasks, let the model plan its next step between
message runs it all, and a human only at the end. -
runs its own checks for accuracy, structure, and writing. Then a
a different model (for example three frontier models from different
procedure Do this, in order: 1. Define good: write the test jobs,
Make only that step an autonomous agent, and pick its pattern
that passes; a stronger one only for the hard step; several in
output as untrusted input; set token and time budgets,
Gather the context you need, then do the work.
task was: {task}. The source is: {source}. Check
the request below. Choose exactly one
returns, check its result; retry
one

practical, build-ready guide to orchestrating AI agents, written for a
magic. They are ordinary software built with AI components.
state? How they share what they know. Each answer sits on
your system sits before you build it. ## 2. Workflow patterns:
publish. ### Parallel Fan out, then bring the results back.
exactly this. A workflow is just a graph of steps: ``` #
patterns: emergent control, routed by the model Use
of failure. If its plan is wrong, everything downstream is
share one conversation. Easy to wire up. Risk: as the
One overwrites the other, and work is lost. The fix is the one
Task 2 to B). No two agents take the same task. - Merges.
context it needs each time. Design the memory, do not hope
and prior results, and keeps
agent gets its own working
evaluation. Many tasks
the same task in isolation, a
test jobs before you commit. ## 7. How to choose One
most: even then, start with a workflow. Lay down the
case Match the pattern to the work: - Known, fixed
parallel workflow. Fan out, bring the results back. -
autonomous agent loop. The agent gathers, acts,
billing, then technical): handoff. Each agent picks who is
published report, anything cited): a worker plus a judge,
autonomy only where it breaks. ## 8. Evaluation: is it
there is a right answer: check it with code (exact match,
many tasks a single call to a capable model matches or
this order. Most of the value
retrieval. What you put in
expensive change and
the loop: repeat
tool calls. A healthy loop is: gather context, take action, re-run the
Intake: one message (from a phone, Slack, or a client) creates
judge, a second agent seeing the work for the first time, reads
providers) building the same deliverable. A copy each, no
including the awkward cases. 2. Start with one call to the
(plan-based, handoff, or group chat) with its failure-mode
parallel when the output is subjective. 8. Add a judge for
not just a turn limit. 11. Keep a human at the end for the pick
Before you finish, verify your output against these checks:
the output for accuracy (every quote and number must
route from this set: {routes}. Return only the route name,
a failed task up to {N} times; then assemble the final result.

machine to read and act on. It is general: it names the
Design them the way you design software, and add
a spectrum. At one end is EXPLICIT CONTROL: you
explicit control, routed in code Use a workflow when
Use it for parts that do not depend on each other:
LangGraph (graph.py) graph.add_edge("draft",
autonomy when you do NOT know the steps, when
wrong. ### Handoff Each agent decides who gets the
conversation grows, cost climbs, details are lost, and
software has always used for parallel work, a copy
Each workspace merges back to the main line. Conflicts are
for it. - Durable state lives OUTSIDE the agent: a task board
only what it needs. - A long shared conversation is NOT
copy of mutable state (section 4). Agents share through
need only one call to a capable model. - Escalate only where it
sandbox each, and let a human or a judge pick the best.
question decides the shape: do you know the steps? - Yes:
steps you do know, run it, and watch where it breaks. Replace only
pipeline (document processing, data extraction, a report with the
Routing or triage (classify a request, send a ticket to the right
verifies, and repeats until done, bounded by a turn limit. - A goal
next; guard against going in circles. - Several candidates to choose
a second model that reviews it; and check the judge. Most real
working? You cannot improve what you cannot measure. Build the
a schema, a passing test . Use a judge when it is a judgement call: a
beats a team of agents at a fraction of the tokens and calls — so
is near the top, not in more agents. 1. Instructions. The
front of the model each turn — the right facts, not more text. Fix
usually the smallest gain; justify each with a measured
gather-and-act until the work is done or a limit is hit. - Verify:
loop, verify the work, with a turn limit so it cannot run
a card on a task board with three columns, NEW, PICKED
the report. After a few fix cycles it is handed back to the
shared state. - Human at the end: the person picks the best
smallest model that could work. Measure quality,
guard. 6. Design the memory: keep durable state
judgement-call outputs, and grade the judge. 9. Optimise
or the approval, and treat the whole thing as
{accuracy, structure, writing}. If you cannot finish
appear in the source, unchanged), structure
nothing else. Request: {request}." Lead or planner
Report which tasks failed." ## 14. Guidelines to

patterns for building agents and, for each kind of application, shows which
intelligence only where it earns its place. ## 1. The two questions
decide everything up front, in code. At the other is EMERGENT
you already know the steps. The route is fixed in code; the model only
summarise ten documents at once, then combine. ### Conditional
"check") # sequential: draft, then check
the path only shows up by trying. Each pattern has a failure mode;
work next: desk, then billing, then tech. Flexible. Risk: the work can
decisions conflict. Many voices, one thread, no owner. Autonomy
each plus a system around it: - A copy each. Give every agent its own
settled at merge time, and the work comes back together. Shared
(NEW, PICKED UP, DONE), saved blocks, files, a database. The agent
memory. Leaning on an
claimed, merged artifacts, not
breaks: a stronger model for
Different models have
explicit control. Use a
the broken step with an
same steps every time): a
handler, choose a tool): a
that splits into sub-goals (build
among (design a page, draft
applications combine these: a
evaluation before you scale
second model reviews the
or a simple task, one call
prompt and the guidelines
what it sees before you
win. ## 10. The agent loop
before finishing, the agent
away. ## 11. Reference
UP, DONE. - Listener: watches
human. During testing, plant a
of the several outputs. That is
tokens, and calls. 3. If one call
outside the agents (a board,
nstructions first, then tools.
software: log every step, cap
within {N} steps, stop and
({required sections}), and
(plan-based): "Break this goal
keep - Start explicit; add

pattern to reach for, how to hold
Every multi-agent design is the
CONTROL: the model decides
ills in each step. Three shapes
Read the request and pick the
graph.add_conditional_edges(
know it before you choose it.
go in circles, agents passing it
buys flexibility and costs
workspace — a git worktree
mutable state is where
reads from it and writes to it; if
ever-growing chat is the
a shared scratchpad. ## 6.
the one hard step, a cheaper
different strengths; trying a
workflow (sequential,
autonomous agent. You end
sequential workflow. Route it
conditional workflow. One
a feature, run a campaign):
copy, propose options): a
ixed pipeline with one
the system. 1. Write the test
output. And check the judge
often wins. Do not add an
each agent follows. Sharper
change how it thinks. 4. Model
nside a single agent is a loop.
checks its own work. -
architecture: a working
the board, picks up a new
fault on purpose (a quote
the only human step. -
s enough, ship it. It often is. 4.
files), and give each agent its
10. Scope the authority
every loop, handle every
report exactly what is
writing ({rules}). List each
nto independent tasks that
emergence only where the

state and memory, and how to
answer to two questions: 1. Who
at run time. Most real systems live in
cover almost everything. ### Sequential One step
route. Use it when the path depends on the input: a
route) # conditional: triage picks the route ``` If you can
Plan-based A lead agent plans the work, hands out
back and forth with no one finishing — so cap the hops (a
control. Reach for it only when a fixed route cannot express the
for code, or a separate branch, row or document version for data.
multi-agent systems break. Isolate it, claim it, merge it.
the agent restarts, the state is still there. - Working
group-chat failure mode: cost climbs, details are lost,
Model choice The model is a replaceable part. Choose it
one for the routine steps. Mixed-model pipelines are
few and choosing beats arguing about which is best. -
parallel, conditional). - No: emergent control. Use
up with a mostly-explicit pipeline and one or two
in code; each step is a model cal
model reads the input and picks
plan-based. A lead plans and
parallel fan-out to several
autonomous step, or a router
jobs first. Decide what good
too, grade the grader against a few known-good and
agent you cannot justify with a measured gain on your
instructions fix more than extra agents do. 2. Tools.
choice. Escalate the one hard step to a stronger model,
Understand it, and bound it. - Gather context: read the
Turn-limit guard: cap the iterations, and raise a clear
system One concrete system that puts the patterns
card, and starts the flow. Everything after is
changed to claim more than it says) and confirm the judge catches
Everything is logged: a live event stream and an explicit per-step
If not, draw the workflow (sequential, parallel, conditional) and
own working copy; add claims and merges for shared
(section 15): least-privilege, sandboxed tools; a human
failure. ## 13. Prompts you can reuse Worker agent:
blocking you. Do not guess." Judge agent: "You
problem and a concrete fix. Do not rewrite the
can be worked in parallel: {goal}.
workflow breaks. - Prefer

mod call just eve add agen with a mea win. -
Cho the mo to the ask cheape that pas esc only the hard step, try
sev whe qual is sub ctiv - Kee dur stat outs the age give eac its own
cop sett conf at mer A ong cha is not me - Use a judg
for judgeme t ca and che the judge. - Define goo with test jobs before you build. - Most gains come from instruc ons and tools, not more age ts. - Kee a human at the end. - Sco e the agent's authori y: least-pri ge, sandb tools a human gate befo e an hing irreve ble; tool output is untrust inp ; cap
tokens and time, not just turns - It is software: name the con and the state, log it, cap the loop handle fa lure. ## 15. Sa ety, per ns and cost An ag nt that calls too acts in the wo treat that s the main isk surface not an afterthough . - Tool output is untrusted input. A docu web pag or API resp nse the age t reads can carry instructions aime at the mo (pr pt
njection). Ke p the system pr and the tas aut ative; rea retr ved ext as data to ana ne er as command to obe - Lea t privilege for ools Give eac agent onl the too and sco es it needs, and sandbox any hing that run code or touc the files stem, network, or a rea acc . A read-only task
gets read only too . - A human gate befo irre e ac s. Spendi mon y, deleting data sending a mes e, d ploying — put an expl approva or a hard allowlist in ron of these, and show wh t will hap before it hap s. - Bud ets not just turn limit Cap tok ns and wa -clock tim per task as well as iter
and stop with a clear "bud exce ed" rath than runn ng up a bill. - Idem cy and retr . Too s fail and time out ma eac acti n saf to r peat (an idemp key or che ck-the ct), retr transien failu s wi backoff, and surf a perm failu inste d of loop ng. An a ent is soft re w h the authority
to act. Scope that auth log every action, and keep a human in front of anything you c nnot und ## Pro ance This page is a writ synth by typograph s-ar org of the field's common, publicly doc ente agent-buil patterns, set out for a machine to read and act on —
not a clai to have invente them. It is se as a Typ aphics Art one-pager: each section carr its own has and the whole age carr the issuer s Ed25519 signature. Verify it at typograph art.org/v the signing key is published at typograph art.org/ and mir on
GitH npm and PyP